

Kubernetes pro vývojáře (praktický přehled)

Cíl dokumentu

Tento dokument shrnuje Kubernetes z pohledu vývojáře (např. Go/Backend), který:

- nasazuje kontejnerové aplikace do clusteru,
- potřebuje rozumět základním objektům (Pod, Deployment, Service, Ingress, ConfigMap, Secret, atd.),
- řeší prostředí (dev/stage/prod), škálování a konfiguraci,
- integruje Kubernetes s dalšími nástroji (CI/CD, observability, storage).

1. Co je Kubernetes

Kubernetes je platforma pro orchestraci kontejnerů, která umožňuje:

- deklarativně popsat stav aplikací a infrastruktury clusteru,
- automaticky plánovat kontejnery (Pods) na nody,
- škálovat, restartovat a upgradovat workloady,
- poskytovat síťovou konektivitu a servisní abstrakce.

Kubernetes se opírá o několik klíčových abstrakcí:

- **Node** – worker (fyzický nebo virtuální stroj), kde běží kontejnery.
- **Pod** – nejmenší nasaditelná jednotka, typicky 1–N kontejnerů.
- **Controller** (Deployment, StatefulSet, DaemonSet, Job) – vysokoúrovňové objekty, které řídí, kolik Podů má běžet a v jaké podobě.
- **Service** – stabilní síťová identita, která směřuje traffic na Pody.
- **Ingress** – HTTP/S vstup do clusteru (typicky za Ingress Controllerem).
- **ConfigMap/Secret** – konfigurace a tajné údaje pro aplikace.

2. Architektura Kubernetes clusteru

2.1 Control plane

Control plane řídí celý cluster, obsahuje komponenty:

- **API server** – REST API, přes které mluví klienti (kubectl, CI/CD, operátory) i interní komponenty.
- **etcd** – distribuovaný key-value store, kde je uložen stav clusteru.

- **Controller manager** – smyčky, které dohlíží, aby běžela správná konfigurace (např. počet Podů v Deploymentu).
- **Scheduler** – plánuje nové Pody na konkrétní nody podle dostupných zdrojů a pravidel.

Pro vývojáře je důležité hlavně API server – vše, co děláš pomocí manifestů či Helm chartů, je jen klient volající API.

2.2 Worker nodey

Na každém worker nodu běží:

- **kubelet** – agent, který komunikuje s API serverem a stará se o Pody na nodu.
- **container runtime** – např. containerd, CRI-O.
- **kube-proxy** (nebo jiný networking plugin) – zajišťuje kubernetes Service networking.

3. Základní objekty

3.1 Pod

Pod je nejmenší jednotka nasazení.

V praxi:

- typicky obsahuje jeden hlavní kontejner a případně sidecar kontejnery (např. logovací agent, proxy),
- sdílí síťový prostor a filesystem (Volumes) mezi kontejnery,
- je efemérní – Kubernetes ho může kdykoliv restartovat / nahradit.

Jako vývojář většinou přímo Pod manifesty nepíšeš – používáš Deployment nebo jiný controller.

3.2 Deployment

Deployment je nejběžnější controller pro stateless aplikace.

Co řeší:

- počet replik (Podů) – horizontální škálování,
- rollout nových verzí (rolling update, canary pattern s pomocí dalších nástrojů),
- rollback na předchozí verze,
- strategii restartu.

Deployment definuje šablonu Podu (`spec.template`), která říká, jak má Pod vypadat: image, porty, resource limity, env proměnné, volume mounty.

3.3 Service

Service poskytuje stabilní endpoint pro Pody,

- má vlastní DNS jméno v clusteru,
- provádí load balancing mezi Pod replikami,
- typy: ClusterIP (interní), NodePort, LoadBalancer, případně další.

Aplikace typicky komunikuje na `http://my-service:port` v rámci namespace.

3.4 Ingress

Ingress definuje HTTP/S routing zvenku do clusteru.

- Ingress Controller implementuje konkrétní chování (např. NGINX, Traefik, AWS ALB Ingress Controller).
- Ingress objekty obsahují pravidla (host/path → Service).
- Podporují TLS terminaci (certifikáty z Let's Encrypt přes cert-manager apod.).

3.5 ConfigMap a Secret

ConfigMap:

- ukládá nekritická konfigurační data (např. aplikační konfigurace, URLs, flagy),
- data jsou stringy / YAML, mountují se jako env proměnné nebo soubory.

Secret:

- ukládá citlivá data (hesla, tokeny, certifikáty),
- jsou base64 enkódované (není to skutečné šifrování, spoléhej se na ochranu storage/API),
- v produkci se často kombinuje s externím secret managerem.

4. Namespaces a izolace

Namespaces slouží k logickému dělení clusteru:

- podle prostředí (dev, stage, prod),
- podle týmů nebo produktů,
- pro systémové komponenty (např. kube-system, monitoring).

Výhody:

- oddělení RBAC (práva na konkrétní namespace),
- oddělený resource quota a limity,
- přehlednější správa.

5. Resource requests a limity

Každý kontejner v Podu může mít:

- **requests** – minimální garantované zdroje (CPU, memory),
- **limits** – maximální povolené zdroje.

Důsledky:

- scheduler umísťuje Pody na nody podle requests,
- při překročení memory limitu může dojít k OOMKill podu,
- při překročení CPU limitu dochází k throttlingu.

Pro Go služby:

- sledovat reálné využití CPU/memory přes metrics (Prometheus/Grafana, Kubernetes metrics),
- nastavit rozumné requests/limits tak, aby služba nepadala ani zbytečně neblokovala zdroje.

6. Storage v Kubernetes

Základní pojmy:

- **Volume** – abstrakce storage připojená do Podu,
- **PersistentVolume (PV)** – reprezentace fyzického/dynamického storage v clusteru,
- **PersistentVolumeClaim (PVC)** – požadavek Podu na konkrétní typ/velikost storage,
- **StorageClass** – definuje typ storage a provisioner (např. EBS, Ceph, NFS, lokální disky).

Pro stateful služby (databáze, fronty) je typicky použitý StatefulSet + PVC.

7. Deployment workflow pro vývojáře

7.1 Manifesty a šablony

Aplikaci nasazuješ pomocí YAML manifestů, často generovaných nástroji jako Helm nebo Kustomize.

- Základní manifesty definují Deployment, Service, ConfigMap, Secret, Ingress.
- Pro více prostředí používáš různé values (Helm), nebo overlays (Kustomize).

7.2 CI/CD

Typický postup:

1. Build Docker image (např. Go service).
2. Push do registry (Docker Hub, ECR, GCR,...).
3. Aktualizace Helm values (tag image, prostředí).

4. `helm upgrade --install` nebo jiné CD.

CI/CD pipeline by měla obsahovat i validaci manifestů (např. `kubectl apply --dry-run`, `kubeval`, `kube-linter`).

8. Observabilita

Pro Kubernetes workloady je klíčová observabilita:

- **Logs** – aplikace loguje do stdout/stderr, Kubernetes je přesměruje na logovací backend (Elastic, Loki, Stackdriver,...).
- **Metrics** – Prometheus + Grafana, případně cloudové metriky.
- **Tracing** – OpenTelemetry + Jaeger/Tempo/Elastic APM.

Z pohledu vývojáře:

- loguj strukturovaně (JSON),
- expose /metrics endpoint pro Prometheus,
- v Go aplikaci propaguj trace ID přes HTTP/gRPC.

9. Bezpečnost a přístup

9.1 RBAC

Role-Based Access Control definuje, kdo může číst/psát jaké objekty.

- Role/ClusterRole + RoleBinding/ClusterRoleBinding,
- omezení práv pro ServiceAccount, které používají Pody.

9.2 Network policies

NetworkPolicy objekty umí omezit komunikaci mezi Pody.

- whitelisting příchozích/odchozích spojení,
- implementace závisí na CNI pluginu (Calico, Cilium,...).

9.3 Pod Security

Moderní clustery používají Pod Security Standards nebo jiné mechanismy k vynucení:

- zákazu běhu jako root,
- omezení privilegovaných kontejnerů,
- omezení hostPath volume.

10. Shrnutí pro tým

Pro vývojáře je nejdůležitější:

- pochopit základní objekty (Pod, Deployment, Service, Ingress, ConfigMap, Secret),
- mít standardizovaný způsob, jak manifesty/Helm charty verzovat a nasazovat,
- mít dobré metriky a logy,
- respektovat limity/requests a bezpečnostní pravidla.